

基于目标优化模型的螺旋路径问题研究

摘 要

“板凳龙”是浙闽地区的传统地方民俗文化活动，研究并优化“板凳龙”表演方案对中国传统文化的创新与传承具有重要意义。本文建立目标优化模型，综合运用微分方程，几何方法，分离轴定理，插值查找等方法研究某时刻板凳龙各节点速度与位置关系、调头方式以及最大速度问题。

针对问题一：首先以螺线中心点为原点建立极坐标系。其次使用**速度分解与微分方程**得到龙头位置表达式，并利用**四阶龙格库塔算法**得到结果。接着基于**余弦定理**，得到相邻节点间辐角的关系，并利用**迭代递推**的方法得到每个节点的位置。然后利用**前向差分**，得到每个节点的速度，并将结果存放于文件 result1.xlsx 中。

针对问题二：我们利用问题一中建立的模型，得到每个节点在具体时刻的位置，并引入**动态碰撞检测方法**。根据节点位置定义板凳边界，设置**碰撞箱与其投影**，并利用**分离轴定理**得到板凳不重叠的充要条件。根据动态碰撞检测，我们求解出在第 **412 秒** 时，龙头所在长凳与其他板凳发生了**碰撞**。同时，我们也记录下该时刻舞龙队的位置和速度，将结果存放于文件 result2.xlsx 中。

针对问题三：对于一个具体的调头空间，首先将问题转化，即要求找到满足板凳碰撞时间恰好等于龙头到达调头空间边界时间的螺距。在问题二模型的基础上，我们可以以螺距为自变量，得到板凳碰撞时间。对于龙头到达调头空间边界时间，我们采用速度路程公式与弧微分，得到螺距与到达时间的关系。同时，为快速确定螺距，我们采用**自适应插值查找算法**，并最终得到：当螺距为 **0.45 m** 时，龙头进入掉头空间边界，同时碰撞发生。

针对问题四：对于一个具体的螺距，首先通过**数学证明**，得到：龙头到达调头空间边界时，立刻进入 S 形曲线即为最优解。其次构建以路径长度最短为目标的**单目标优化模型**。然后基于一阶导数信息的**线性搜索算法**不断更新迭代点来减小目标函数的值，并依据 **Armijo-Goldstein 准则**提高搜索效率，最终得到调头路线，并将给定时刻舞龙队的位置与速度存放到文件 result4.xlsx 中。

针对问题五：根据问题四的轨迹，首先通过几何关系建模，解得龙头运动轨迹的参数方程。其次，利用**牛顿法**结合曲线的**弧长参数化**，在给定方向上更新迭代点从而逼近相邻节点，并利用递推求解全部节点位置。然后构建以在约束下最大化龙头速度的单目标约束优化模型，进行一系列仿真。最终得到满足速度条件的最大龙头速度为 **1.3124 m/s**。

关键词：螺旋路径 目标优化 龙格库塔算法 插值查找算法 线性搜索算法

一、 问题重述

1.1 问题背景

龙舞作为华夏传统文化，象征吉祥幸福，气势磅礴，广泛流传，是节庆民俗代表和全民健身的重要项目。然而，随着现代化进程，龙舞文化的传承面临严峻挑战：资金不足、群众基础薄弱、传承方式单一等一系列问题，导致这一传统文化逐渐衰退。板凳龙作为龙舞中的一种，虽具有鲜明的地方特色，但如何在现代社会中保持其文化生命力并与时俱进，也成为了一大挑战。

因而，通过数学建模优化舞龙队的行进路径与空间利用效率，以提升板凳龙的观赏性和表现效果。针对板凳龙表演中的技术性问题，通过精确计算舞龙队各部分的位置和速度，提供优化的表演方案，是提升观赏性、增强表演效果，并推动龙舞文化在新时代背景下传承与创新的重要手段^[1]。

1.2 问题重述

问题一：根据题目给定的盘入螺线示意图，在螺距为 55 cm 的等距螺线条件下，龙头以 1 m/s 的速度盘入，计算从初始时刻到 300 秒内每秒龙头、龙身和龙尾各节前把手及龙尾后把手的位置和速度，特别是在 0 秒、60 秒、120 秒等特定时刻记录其位置和速度，并将结果保存到文件 result1.xlsx 中。

问题二：舞龙队仍然沿问题 1 给定的螺线盘入，在确保板凳之间不发生碰撞的前提下，确定舞龙队能够顺利盘入的终止时刻，给出此时龙头、龙身（第 1、51、101、151、201 节）前把手及龙尾后把手的位置和速度，并将结果保存到文件 result2.xlsx 中，供进一步分析。

问题三：在舞龙队从顺时针盘入切换为逆时针盘出的过程中，调头空间以螺线中心为圆心，直径为 9 米，计算最小螺距，使龙头前把手能顺利到达调头空间的边界。

问题四：根据题目给定的要求，在调头过程中，舞龙队的调头路径由两段相切的圆弧组成的 S 形曲线构成。在此条件下，探讨能否调整圆弧，使调头路径变短，同时保持各部分相切，计算舞龙队在调头过程中的位置与速度，并将结果保存到文件 result4.xlsx 中。

问题五：在舞龙队沿问题 4 设置的调头路径行进时，确定龙头的最大行进速度，确保各节把手的速度不超过 2 m/s，优化整体表演的速度和流畅性。

二、 问题分析

2.1 问题一的分析

在问题一中，要求在沿等距螺线顺时针运动且龙头行进速度不变的情况下，建立舞龙队速度与位置关于时间的数学模型，并在具体时间下得到具体节点的速度与位置。首先，以圆心为中心建立**极坐标系**，然后通过求解**微分方程**来确定每个时刻龙头的位置与速度。在此基础上利用**余弦定理**，得到相邻节点辐角之间的

递推关系，据此来得到每个时刻具体节点的位置与速度。至此，模型建立完成，将具体的时间与节点带入模型，从而得到相应的位置与速度。

2.2 问题二的分析

在问题二中，要求我们不更改路线与速度，找到碰撞的时刻。由问题一建立的模型可知，舞龙队的位置与速度仅由时间来决定。于是对于问题二的解决，我们需要沿用问题一的模型，运行仿真以模拟各节点运动。对于碰撞条件的判断，我们使用**分离轴定理**，通过在仿真中检查每一时刻，板凳两两之间是否有位置重叠，从而来判断是否发生碰撞。由此得到第一碰撞时间并获取该时刻各节的位置与速度。

2.3 问题三的分析

在问题三中，给定一个具体的掉头空间，要求确定最小螺距，使得龙头能进入掉头空间。这里可以转化为：在某一螺距条件下，板凳碰撞的最小时间不小于龙头到达给定掉头空间所需的时间。因此，我们可以继续套用问题一与问题二的模型，在一系列给定的螺距下，计算板凳发生碰撞的时间与到达掉头空间边缘的时间并进行比较。对于螺距的选择，我们使用**自适应的插值查找**，根据到达的时间插值，改变查找区间，从而确定最小螺距。

2.4 问题四的分析

在问题四中，给定一个具体螺距，要求在问题三给定的空间内寻找最佳的调头方案。为保证良好的观赏性以及移动的高效性，调头路径应由两端半径比为 2:1 的相切的圆弧组成；两端应与盘入、盘出螺线相切；盘入与盘出螺线应成中心对称；需要在调头区域内完成掉头；调头曲线应尽量短。我们通过对问题**几何建模**，表达约束条件，将问题转化为**约束优化问题**，并使用 **Armijo 准则**进行线搜索，最终得到最优路线。

2.5 问题五的分析

在问题五中，舞龙队沿问题四最短路线行进，在舞龙队各把手速度不超过 2 米每秒的约束条件下，要求找到最大的龙头行进速度。我们首先利用几何关系得到节点位置，并利用弧长参数化表示行进曲线。由于曲线的复杂性，对于每个节点，我们使用**牛顿法**逼近相邻节点位置，然后根据龙头速度**递推**得到各节点速度。最后，我们使用**迭代**方法求得允许的最大龙头速度。

三、 模型假设

1. 不考虑板凳垂直方向的运动，只考虑二维平面运动。
2. 将把手视为质点，认为始终在螺线上而不发生偏移
3. 不考虑人为操作错误与周围环境的影响。

四、 符号说明

符号	说明	单位
ρ	极径	m
θ_i	辐角	/
d	螺距	m
l_0	板凳长度	m
l_1	孔与板头的距离	m
e_i	碰撞箱的边方向向量	/
P_i	碰撞箱第 <i>i</i> 个顶点	/
$Axis_i$	第 <i>i</i> 条分离轴	/
v_0	龙头速度	m/s
S	龙头走过的弧长	m
θ_{end}	到达调头空间时龙头的辐角	/
t_{stop}	到达边界所需时间	s
$t_{collision}$	发生碰撞的时间	s
r	包络圆半径	m

五、 模型的建立与求解

5.1 问题一模型建立与求解

5.1.1 微分方程模型建立

根据题目对旋线的描述可以知道，舞龙队沿**等距螺线**(阿基米德螺线)行进。为描述其几何特点，以螺线中心为原点建立极坐标系，螺线方程为：

$$\rho = \frac{d}{2\pi}\theta = b\theta \quad (1)$$

其中螺距 d 在本文中为 0.55m。我们从龙头开始分析舞龙队的运动。龙头的速度可分解为径向速度和切向速度：

$$v_r = \frac{d\rho}{dt} \quad (2)$$

$$v_\theta = \rho \frac{d\theta}{dt} \quad (3)$$

$$\vec{v} = v_r \hat{r} + v_\theta \hat{\theta} \quad (4)$$

由题目已知，龙头行进速度为常数 $v_0 = -1$ m/s。综上我们得到微分方程：

$$\frac{d\theta}{dt} \sqrt{\theta^2 + 1} = \frac{2\pi v_0}{d} \quad (5)$$

5.1.2 四阶龙格库塔算法分析

为了求解上述得到的常微分方程，本文使用精度较高的四阶龙格库塔法(Runge-Kutta)。

1. 四阶龙格库塔算法分析

对于形如 (6) 的一阶微分方程初始值问题，四阶龙格库塔法的求解递推公式如 (7) 所示：

$$\begin{cases} \dot{y} = f(t, y) \\ y(t_0) = y_0 \end{cases} \quad (6)$$

其中， y_0 为初始状态， t_0 为已知时间，函数 $f(t, y)$ 描述了 y 随时间 t 的变化关系。

$$\begin{cases} t_{n+1} = t_n + h \\ k_1 = f(y_n, t_n) \\ k_2 = f\left(y_n + \frac{h}{2}k_1, t_n + \frac{h}{2}\right) \\ k_3 = f\left(y_n + \frac{h}{2}k_2, t_n + \frac{h}{2}\right) \\ k_4 = f(y_n + hk_3, t_n + h) \\ y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{cases} \quad (7)$$

任意时刻的值 $y(n)$ 是通过前一时刻的值 $y(n-1)$ 与时间步长 h 以及近似斜率的乘积来确定的。对于每一个时间步，其估计的斜率为以下各个斜率的加权平均：

- k_1 ：起始时刻 t_n 的斜率；
- k_2 ：时间中点的斜率。使用斜率 k_1 估算在 $t_n + \frac{h}{2}$ 处的 y 值；
- k_3 ：时间中点的斜率，基于 k_2 的估计；
- k_4 ：终点时刻 $t_n + h$ 的斜率，使用 k_3 估算。

易知，通过对这些斜率进行加权平均，时间中点斜率权重最大^[2]。根据 (7) 的递推公式即可求得 $y(1), y(2), \dots, y(N)$ 。

2. 利用龙格库塔算法求解模型

Step 1: 初值条件的确定：根据题目描述可知，初始时龙头位于螺线第 16 圈处，根据极坐标可得 $\theta_0 = 32\pi$ 。

Step 2: 微分方程求解：由于本问题求解的方程为一阶一元微分方程，因此可直接利用算法进行求解，求解结果如图所示：

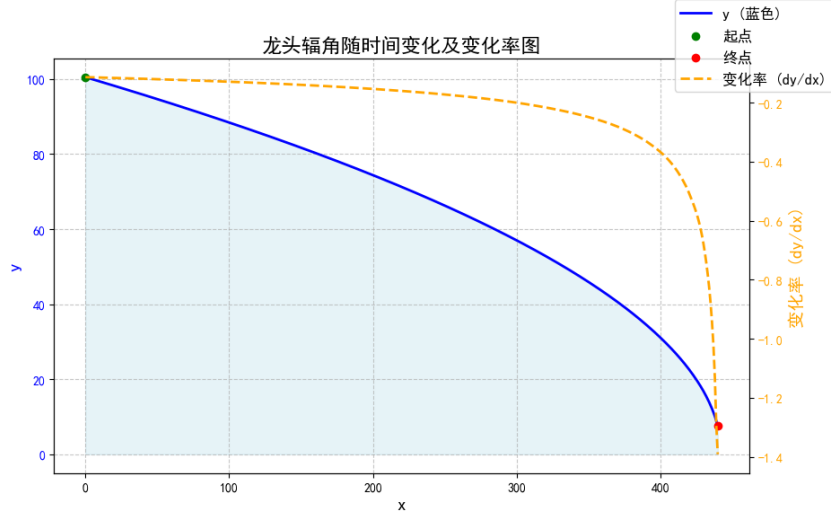


图 1: 龙头倾角随时间变化及变化率曲线图

5.1.3 基于余弦定理建立递推关系

通过微分方程的求解，我们已获得任意时刻龙头的位置与速度。为了获得任意节点在任意时刻的位置与速度，我们需要对舞龙队节点位置进行几何建模，并通过第 i 个节点的位置递推得到第 $i+1$ 个节点位置。具体操作如下：

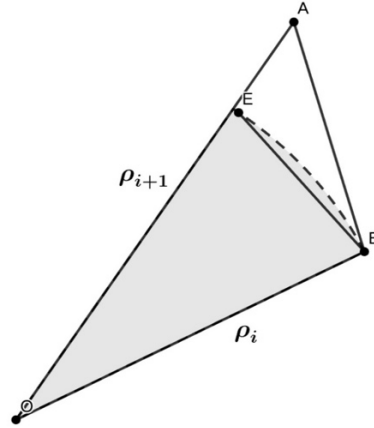


图 2: 舞龙队节点位置递推几何模型示意图

在上图中，O 为极坐标系的坐标原点，点 A 与 B 分别为同一板凳的两个节点。我们以 O 点为圆心，过 B 点画弧并与 AO 边交于点 E，使得 $OE = OB$ 。根据极坐标定义以及三角形边角关系，可得：

$$OB = b\theta_i \quad (8)$$

$$OA = b\theta_{i+1} \quad (9)$$

$$AB = 2b\theta_i \sin \frac{\theta_{i+1} - \theta_i}{2} \quad (10)$$

至此，我们已知相关的边长与角度，下面可通过余弦定理将角与边联系起来，从而得到 θ_i 和 θ_{i+1} 的关系。

余弦定理是描述三角形中三边长度与一个角的余弦值关系的数学定理，它指出：对于任意三角形，任何一边的平方等于其他两边平方的和减去这两边与它们夹角的余弦的积的两倍。我们在 ΔABE 中运用余弦定理，可得：

$$AE^2 + BE^2 - 2AE \cdot BE \cos \frac{\pi + \theta_{i+1} - \theta_i}{2} = (l_0 - 2l_1)^2 \quad (11)$$

联立 (8) (9) (10) (11) 式，我们可以求得 $\theta_i(t)$ 的递推关系式如下：

$$b^2 \Delta \theta^2 + \left(2b\theta_i \sin \frac{\Delta \theta}{2} \right)^2 + 4b^2 \Delta \theta \theta_i \sin^2 \frac{\Delta \theta}{2} = (l_0 - 2l_1)^2 \quad (12)$$

其中， $\Delta \theta = \theta_{i+1} - \theta_i$ 。

根据该递推方法，我们通过**迭代递推**方法求解整个板凳龙队伍各个节点的辐角。对于每一时步，我们将微分方程解得的龙头辐角代入，通过**二分法**逼近解得相邻节点辐角，**回代、递推**，直到求出整个队伍辐角值。以下是计算得到的队伍第 400 秒时的队形示意图：

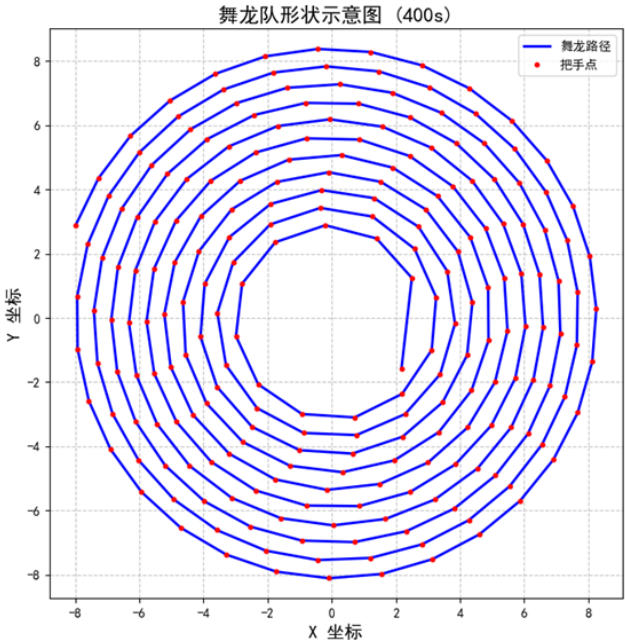


图 3：舞龙队形螺旋状态示意图（400 秒）

5.1.4 各节点位置与速度的求解

根据上述模型的计算结果，我们将时间参数 t 代入模型，并进行**迭代求解**，得到了不同时间点下龙头、龙身各节点及龙尾的把手中心位置变化情况，结果如下表所示：

表 1：不同时间点龙头、龙身和龙尾各节把手位置变化表

节点	0s	60s	120s	180s	240s	300s
x	8.800000	5.696771	-4.323163	-2.604788	2.089625	4.682438
y	0.000000	-5.870110	-6.139404	6.251070	-5.564527	1.696909
1	8.363824	7.394584	-1.728114	-4.974456	4.471029	3.019873

1	2.826544	-3.568581	-7.341619	4.649734	-3.980698	4.024027
51	-9.518732	-8.643951	-5.319316	3.248277	5.663038	-6.206356
51	1.341138	2.676083	6.561924	7.091343	-4.271623	1.134387
101	2.913982	5.569731	5.121565	1.514679	-5.333860	-5.845313
101	-9.918312	-8.081992	-7.719271	-8.544645	-6.028086	4.485028
151	10.861727	6.790955	2.668332	1.392445	3.471562	7.368070
151	1.828752	8.042518	9.651839	9.371709	8.197658	3.803157
201	4.555107	-6.503684	-10.584218	-9.440332	-7.789044	-7.821457
201	10.725116	9.108131	1.646331	-3.886858	-5.748553	-4.694845
x	-5.305446	7.254142	10.990444	7.654771	3.749884	2.440958
y	-10.676583	-8.887849	0.554243	7.210658	9.274466	9.144487

为了进一步分析运动情况，我们使用前向差分法基于相邻时间点的节点位置，计算出对应时间点的速度。具体计算公式如下：

$$\begin{aligned}
 u_{i,t} &= \frac{u_{i,t+1} - u_{i,t}}{\Delta t} \\
 v_{i,t} &= \frac{v_{i,t+1} - v_{i,t}}{\Delta t}
 \end{aligned}
 \tag{13}$$

其中， $\vec{v}_{i,t} = u_{i,t} \cdot \hat{i} + v_{i,t} \cdot \hat{j}$ 表示速度向量的分解。基于此方法，求得各时间点的速度变化，结果如下表所示：

表 2：不同时间点龙头、龙身和龙尾各节把手速度变化表						
	0s	60s	120s	180s	240s	300s
头	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
1	4.681011	1.000302	0.998798	0.998375	0.997539	0.994758
51	5.377497	1.000442	0.998995	0.998679	0.997971	0.995517
101	5.170594	1.000538	0.999124	0.998847	0.998207	0.995873
151	4.216332	1.000610	0.999215	0.998961	0.998354	0.996071
201	5.689166	1.000676	0.999288	0.999045	0.998458	0.996206
尾	2.488629	1.000697	0.999313	0.999075	0.998494	0.996251

5.2 问题二模型建立与求解

在问题二中，我们沿用问题一中的行动路径，同时需计算舞龙队发生碰撞的时刻及该时刻各节点的速度与位置。根据问题一，舞龙队的速度与位置仅依赖于时间。因此，只需确定板凳之间发生碰撞的条件，进行判定后，再利用问题一的模型求解即可。

5.2.1 动态碰撞检测方法

在动态碰撞检测方法中，我们首先定义节点的运动轨迹。接着，在每一时步中根据节点位置定义板凳边界，设置碰撞箱。之后，在运行仿真时，对每一时步所有非相邻的碰撞箱检测是否存在重叠。

1) 检测板凳是否重叠的数学定理

分离轴定理 (Separating Axis Theorem, 简称 SAT) 是一种用于检测两个凸多边形是否相交的数学定理。该定理指出，如果存在一条轴，使得两个凸多边形在该轴上的投影不重叠，那么这两个多边形在三维空间中是不相交的。在二维空间中，对于凸多边形，这条轴可以是多边形的边向量与多边形的法线向量的组合。

在我们的仿真中，我们对每一时间步中所有非相邻板凳运用分离轴定理进行检验，步进直至第一次碰撞箱重合，也即碰撞发生。

2) 定义碰撞箱与投影

一般地，我们研究由第 i 个和第 $i+1$ 个节点固定的板凳。其坐标及方向向量为：

$$\begin{cases} x_i = b\theta\cos \theta_i \\ y_i = b\theta\sin \theta_i \\ \mathbf{e}_1 = (x_i - x_{i+1}, y_i - y_{i+1}) \\ \mathbf{e}_2 = (-(y_i - y_{i+1}), x_i - x_{i+1}) \end{cases} \quad (14)$$

由此，我们可以找到矩形各顶点的坐标：

$$\begin{cases} P_1 = (x_i, y_i) - l_1 \mathbf{e}_1 + \frac{l_2}{2} \mathbf{e}_2 \\ P_2 = (x_i, y_i) - l_1 \mathbf{e}_1 - \frac{l_2}{2} \mathbf{e}_2 \\ P_3 = (x_i + 1, y_i + 1) + l_1 \mathbf{e}_1 + \frac{l_2}{2} \mathbf{e}_2 \\ P_4 = (x_i + 1, y_i + 1) + l_1 \mathbf{e}_1 - \frac{l_2}{2} \mathbf{e}_2 \end{cases} \quad (15)$$

对应的边向量为：

$$\begin{cases} \mathbf{Edge}_1 = (P_1 - P_0) \\ \mathbf{Edge}_2 = (P_3 - P_0) \end{cases} \quad (16)$$

分离轴为边向量的法向量，因此满足：

$$\mathbf{Axis}_1 \cdot \mathbf{Edge}_1 = \mathbf{Axis}_2 \cdot \mathbf{Edge}_2 = 0 \quad (17)$$

对于每个顶点 $P = (a, b)$ ，其在分离轴 $\mathbf{Axis} = (a_x, a_y)$ 上的投影为：

$$\text{Proj}(P_i, \mathbf{Axis}) = a \cdot a_x + b \cdot a_y \quad (18)$$

对应的投影区间为：

$$\left[\min_i \text{Proj}(P_i, \mathbf{Axis}), \max_i \text{Proj}(P_i, \mathbf{Axis}) \right] \quad (19)$$

3) 板凳不重叠的条件

根据分离轴定理，两凸多边形在某分离轴上分离的充要条件为在此分离轴上的投影区间不重叠，即：

$$\begin{aligned} \max_i \text{Proj}(P_i, \text{Axis}) < \min_j \text{Proj}(Q_j, \text{Axis}) \text{ or} \\ \max_j \text{Proj}(Q_j, \text{Axis}) < \min_i \text{Proj}(P_i, \text{Axis}) \end{aligned} \quad (20)$$

对于矩形，两边向量的法向量即全部可能的分离轴。当两矩阵在至少一条分离轴上的投影没有重合，则它们一定分离。反之则存在重合。

5.2.2 时间步进检测

对每一个时间步，我们对问题一中的各节点运动轨迹和距离进行动态碰撞检测。在我们的仿真中，舞龙队伍在第 412 秒发生了碰撞。

通过如下示意图，我们可以看到，第 412 秒时，龙头所在长凳与其他板凳发生了碰撞，此时队伍不能再继续盘入。

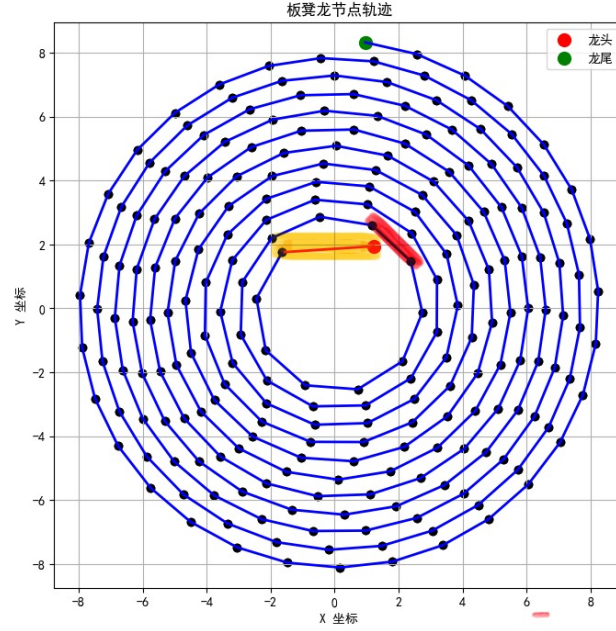


图 4: 板凳龙节点轨迹示意图

此时各节点的速度大小如图所示。可以看到越靠近外侧，速度越小，速度变化越慢。

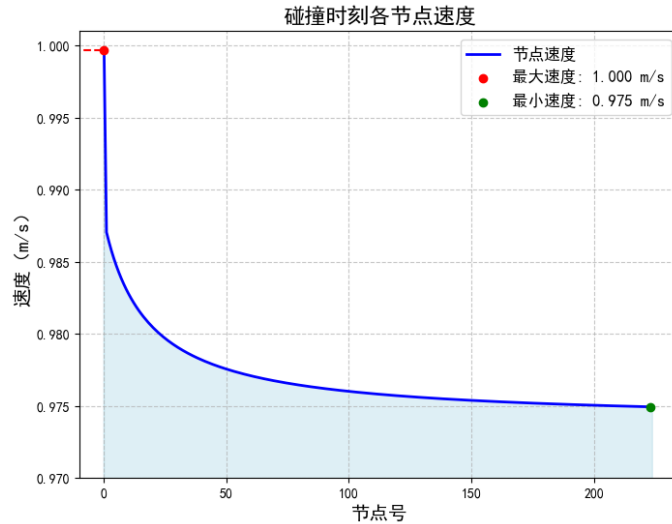


图 5: 碰撞时刻各节点速度分布图

部分的具体数值如下表所示:

表 3: 舞龙队螺线盘入终止时刻及位置速度数据表

	横坐标 x (m)	纵坐标 y (m)	速度 (m/s)
龙头	1.560832	1.649862	0.970424
第 1 节龙身	-1.275441	2.017497	0.965416
第 51 节龙身	1.699996	4.170442	0.971234
第 101 节龙身	-0.977632	-5.816333	0.972144
第 151 节龙身	0.525241	-6.999377	0.972516
第 201 节龙身	-7.944635	-0.788402	0.972721
龙尾 (后)	1.396402	8.255567	0.972783

5.3 问题三模型建立与求解

根据题意, 问题三中要求龙头前把手能够沿着相应的螺线盘入到调头空间的边界, 等价于在盘入调头空间边界之前没有发生碰撞, 亦即发生碰撞的时间步不小于到达调头空间边界的时间步。临界情况即到达调头空间和碰撞同时发生。因此, 我们需要分别求解龙头到达边界所需时间与碰撞的时间步, 当时间差最小时, 螺线即为所需。

5.3.1 目标函数建立

发生碰撞前, 已知龙头速度固定为 v_0 , 经过时间 t , 则龙头走过的弧长 S 为:

$$S = v_0 t \quad (21)$$

同时, 弧长也可由积分得到。根据弧微分可知:

$$ds = \sqrt{(d\rho)^2 + (\rho d\theta)^2} \quad (22)$$

代入螺线方程 $\rho = \frac{d\theta}{2\pi}$ ，可得到下列方程，此时螺距 d 为参数：

$$ds = \sqrt{\left(\frac{d}{2\pi}\right)^2 + \left(\frac{d\theta}{2\pi}\right)^2} d\theta \quad (23)$$

接着对两侧积分，即可获得从开始至进入边界，龙头走过的总路程：

$$S = \frac{d}{2\pi} \int_{\theta_{end}}^{\theta_0} \sqrt{\theta^2 + 1} d\theta \quad (24)$$

其中 θ_{end} 为到达调头空间时龙头的辐角，且已知调头空间的直径为 9m，则对 θ_{end} 的限制条件为：

$$4.5 = \frac{d}{2\pi} \theta_{end} \quad (25)$$

因此，综合上述式子可得，从开始至进入边界所需要的时间 t_{stop} 为：

$$t_{stop} = \frac{d}{4\pi v_0} \left[\ln(\sqrt{\theta^2 + 1} + \theta) + \theta\sqrt{\theta^2 + 1} \right] \Big|_{\theta_{end}}^{\theta_0} \quad (26)$$

而碰撞发生的时间步可由问题二中模型的**动态监测**得到。综上所述，题目所求螺距 d_{opt} 即为：

$$d_{opt} = \operatorname{argmin} \left| \frac{d}{4\pi v_0} \left[\ln(\sqrt{\theta^2 + 1} + \theta) + \theta\sqrt{\theta^2 + 1} \right] \Big|_{\frac{9\pi}{d}}^{\theta_0} - t_{collision}(d) \right| \quad (27)$$

5.3.2 自适应的插值查找

在问题三中，由于得到临界螺距需要求解板凳碰撞时间与龙头到达调头空间的时间，若采用循环遍历法分别遍历，则时间成本较高。因此我们使用自适应的插值查找来获取所需结果^[3]。

插值查找 (Interpolation search) 是利用插值公式来计算猜测搜索键值的位置。具体键值的公式为：

$$mid = low + \frac{key - a[low]}{a[high] - a[low]} * (high - low) \quad (28)$$

在本题中，该算法的具体流程如下：

Step 1: 初始化遍历区间: 设定螺距最小为 0m, 最大为 1m。

Step 2: 计算时间差: 计算给定区域最小螺距与最大螺距对应的板凳碰撞时间 t_1 与龙头到达调头空间的时间 t_2 , 并作差得到 $\Delta t = t_1 - t_2$ 。则最小值与最大值的时间差分别为 Δt_{min} 与 Δt_{max} 。

Step 3: 得到下一个计算的螺距: 由于所需值 $\Delta t = 0$, 所以下一个螺距值即为 $min - \frac{\Delta t_{min}}{\Delta t_{max} - \Delta t_{min}} * (max - min)$ 。

($max - min$)。

Step 4: 重新计算遍历区间: 计算新得到的螺距对应的板凳碰撞时间与龙头到达调头空间的时间并作差, 若该值小于 0, 则将最小螺距设置为新得到的螺距; 若该值大于 0, 则将最大螺距设置为新得到的螺距。

Step 5: 迭代至结束: 重复上述过程直至最小螺距大于等于最大螺距, 由于可能不存在时间差为 0 对应的螺距值, 则最后所需的螺距值即为最小螺距的值。

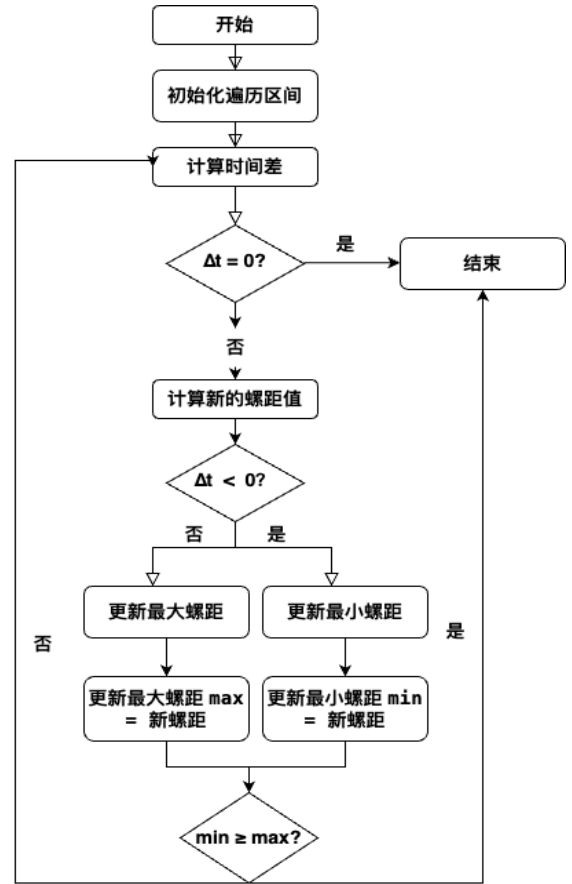


图 6: 插值查找算法流程图

5.3.3 优化搜索结果

通过自适应插值查找, 可得当螺距为 **0.45 m** 时, 龙头在第 **218 秒** 进入调头空间边界, 并同时发生**碰撞**。以下为临界时刻的队形示意图:

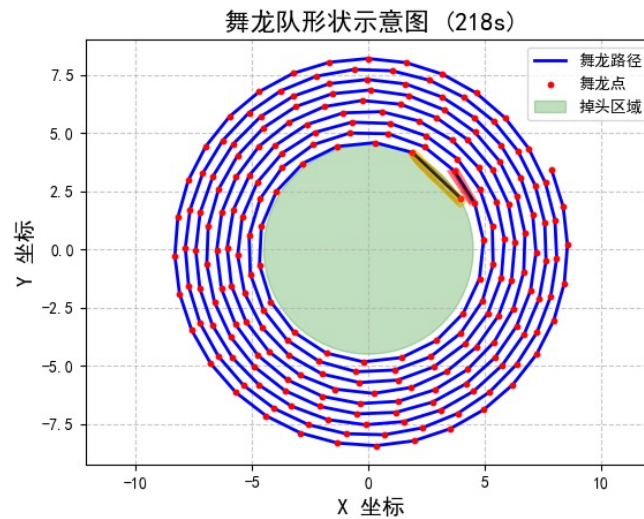


图 7: 舞龙队在 218s 时的形状示意图

5.4 问题四模型建立与求解

本问题要求在约束条件下，寻找最优的调头路径。由于题目中给出的约束条件众多，曲线自由度并不高，因此，有必要进行详细的几何建模，找到合适的决策域，进而求解优化问题得到最优路线。

5.4.1 几何关系建模

题目中提出的对曲线的约束条件可划分为以下几条：

- 调头路径由两端相切的圆弧组成
- 圆弧半径比为 2 : 1
- 路径两端与盘入、盘出螺线相切
- 在调头空间内完成调头
- 盘入弧线与盘出弧线中心对称

为了方便满足所有约束条件，我们以两个圆弧的圆心位置 (x_1, y_1) 和 (x_2, y_2) 表征曲线，即一旦圆心位置确定，曲线即唯一确定。

为保证圆弧相切且半径比为 2 : 1，两圆半径应满足以下条件：

$$\begin{cases} x_1^2 + (r - y_1)^2 = \frac{1}{9}[(x_2 - x_1)^2 + (y_2 - y_1)^2] \\ x_2^2 + (r + y_2)^2 = \frac{4}{9}[(x_2 - x_1)^2 + (y_2 - y_1)^2] \end{cases} \quad (29)$$

同时，根据题目中约束条件可得，两圆要分别与盘入、盘出螺线相切，圆心应满足：

$$\begin{cases} y_1 = r + kx_1 \\ y_2 = kx_2 - r \end{cases} \quad (30)$$

其中， k 为螺线在切入和切出处切线的斜率，其表达式为：

$$k = \frac{\sin \theta + \theta \cos \theta}{\cos \theta - \theta \sin \theta} \quad (31)$$

为简化计算，我们取点 $(0, 4.5)$ 处为切点，解得 $k = \frac{2}{\pi}$ 。

综上，我们已经得到了切入点和切出点的斜率， x_1, y_1, x_2, y_2 应满足以下方程组的解：

$$\begin{cases} x_1^2 + (r - y_1)^2 = \frac{1}{9}[(x_2 - x_1)^2 + (y_2 - y_1)^2] \\ x_2^2 + (r + y_2)^2 = \frac{4}{9}[(x_2 - x_1)^2 + (y_2 - y_1)^2] \\ y_1 = \frac{2}{\pi} + r \\ y_2 = \frac{2}{\pi} - r \end{cases} \quad (32)$$

不难发现,该方程组在实数范围内存在唯一解。由于我们将切点设置在(0,4.5),还需要将两个圆心在坐标系中旋转角度 ϕ , 以确保切入点位于盘入螺线的末端:

$$\phi = \theta_{end} - \frac{\pi}{2} \quad (33)$$

据此角度, 定义旋转矩阵 $R(\theta)$ 为:

$$R(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix} \quad (34)$$

在变换后的坐标系内, S 弯弧线的两圆心应为:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = R(\theta) \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x \cos(\theta) - y \sin(\theta) \\ x \sin(\theta) + y \cos(\theta) \end{pmatrix} \quad (35)$$

至此, 圆心的位置经过旋转后得到变换坐标。

5.4.2 路径长度目标函数构建

在调头区域内, 曲线总长度由两部分构成: 第一部分是半径为 r 的小圆内的 S 形弯曲路径, 第二部分是两端位于调头区域外小圆的螺线弧段。对于螺线, 从进入调头区域到 S 弯起点的弧长为:

$$S' = \int_{\theta_r}^{\theta_{end}} \sqrt{\theta^2 + 1} d\theta \quad (36)$$

对于两段圆弧, 总弧长可以表示为:

$$C = \gamma \cdot \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (37)$$

其中, γ 为两圆心连线与螺线切线法向方向的夹角。

综上, 目标函数确定为:

$$L(r) = \cos^{-1} \left(\frac{\vec{l} \cdot C_1 O_2}{|\vec{l}|} \right) + \frac{d}{2\pi v_0} \left[\ln \left(\sqrt{\theta^2 + 1} + \theta \right) + \theta \sqrt{\theta^2 + 1} \right] \Big|_{\theta_{end}}^{\theta_0} \quad (38)$$

舞龙队的调头示意图如下所示：

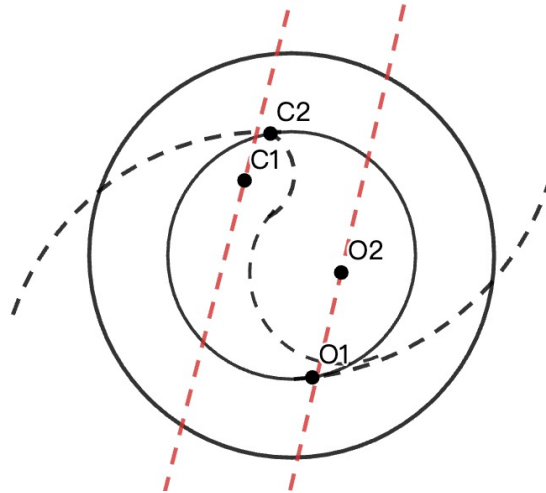


图 8: 舞龙队调头路径示意图

根据题目条件，盘入弧线与盘出弧线中心对称。因此，经过螺线切入点，并垂直于螺线的切线方向的两条红色虚线平行。由此可知，S 弯两段圆弧的圆心应分别位于两条红色虚线上，才能满足约束条件 3，即路径两端与盘入、盘出螺线相切。

5.4.3 求解最优半径

根据题目的要求，我们需要确定从螺旋曲线进入 S 形曲线的最佳时机，即找到 S 形曲线的包络圆的最优半径。

为找到最优半径，我们采用基于一阶导数信息的线性搜索方法以最小化目标函数。其核心目标是通过不断更新迭代点来逐步减小目标函数的值：

$$\begin{aligned} r_{k+1} &= r_k + \alpha_k p_k \\ \alpha_k &= \arg \min_{\alpha} l(r_k + \alpha p_k) \end{aligned} \quad (39)$$

然而，考虑到目标函数的复杂性和特点，使用简单线性搜索效率较低。为了提高计算效率，我们依据 **Armijo-Goldstein 准则**，在每次迭代中动态调整线搜索的步长，以保证每次迭代都能有效降低目标函数值。

具体而言，在每次迭代中，我们检查 **Armijo 条件** 和 **曲率条件**：

$$\begin{aligned} l(r_k + \alpha_k p_k) &\leq l(r_k) + c_1 \alpha_k \nabla l(r_k)^T p_k \\ \nabla l(r_k + \alpha_k p_k)^T p_k &\geq c_2 \nabla l(r_k)^T p_k \end{aligned} \quad (40)$$

其中，在每次迭代中沿着搜索方向前进时调整步长，从而保证目标函数的下降量足够大，以避免计算复杂的精确线搜索。

搜索结果表明，当半径为 4.5 米的时候，调头路线取得最小值，结果如下所示：

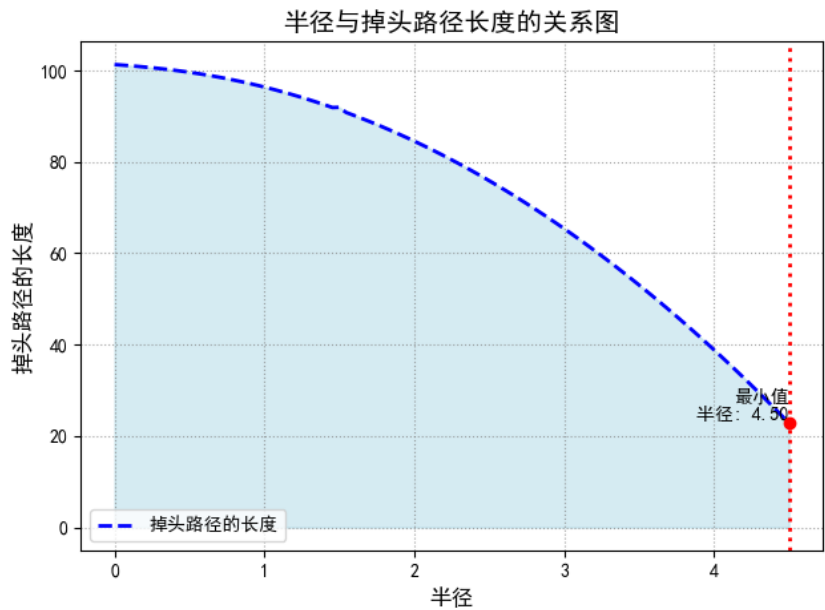


图 9： 包络圆半径与舞龙队调头路径长度关系图

舞龙各节在不同时间点的速度和位置变化详见下表：

表 4： 不同时间点舞龙各节速度变化表

(m/s)	-100s	-50s	0s	50s	100s
龙头 y	0.999892	0.987437	1.006372	0.986326	1.004588
第 1 节龙身	0.998573	0.985860	1.002456	0.983995	1.004577
第 51 节龙身	0.998644	0.998899	0.996595	0.999643	0.989481
第 101 节龙身	1.000073	0.999122	0.998654	1.007529	0.987767
第 151 节龙身	0.999317	1.000261	0.999549	0.998438	1.001327
第 201 节龙身	0.998546	0.998957	1.000137	0.999394	0.998534
龙尾（后）	0.998894	0.998473	0.999432	1.000111	0.999241

表 5： 不同时间点舞龙各节位置变化表

(m)	-100s	-50s	0s	50s	100s
头 x	7.918109	6.700851	-2.984056	-6.760713	-8.422802
1	6.982160	6.083686	-1.296892	-6.252806	-8.463808
51	4.487637	-6.220615	-6.276673	2.149162	5.459516
101	9.905831	-8.922251	-6.769233	-1.649588	-4.992695
151	11.274709	-3.238171	-8.266435	6.788770	8.502495
201	-9.140893	11.876852	-12.641731	5.439364	8.255494
尾 x	13.883112	-11.208664	5.399006	4.324054	-11.038557
头 y	3.379486	1.459898	-3.338372	0.507446	-1.445844
1	5.142859	3.330172	-4.410926	2.408224	0.558065

51	9.498864	6.679967	4.093649	4.969772	2.659040
101	6.886466	6.235070	-6.753737	-7.847438	-3.533599
151	-7.322982	11.966101	-7.633317	-7.311804	0.445778
201	-11.503831	6.913779	1.408682	-10.253533	6.285670
尾 y	-6.140394	8.819211	-12.133379	11.427808	-0.611223

5.5 问题五模型建立与求解

5.5.1 几何关系建模

在本问题中，我们首先通过求解龙头的运动轨迹，利用几何关系递推得到所有节点的位置。结合问题四中求得的最短路径，我们可以表示龙头运动轨迹的参数方程为：

$$\begin{cases} x = b\theta_t \cos \theta_t \\ y = b\theta_t \sin \theta_t \end{cases} \quad t < 0 \\
 \begin{cases} x = x_1 + r_1 \cos(2 - v_0 t) \\ y = y_1 + r_1 \sin(2 - v_0 t) \end{cases} \quad 0 < t < \frac{\alpha r_1}{v_0} = t_1 \\
 \begin{cases} x = x_2 + r_2 \cos(\beta + v_0 t) \\ y = y_2 + r_2 \sin(\beta + v_0 t) \end{cases} \quad \frac{\alpha r_1}{v_0} < t < \frac{\alpha r_1 + \beta r_2}{v_0} = t_2 \\
 \begin{cases} x = -b\theta_{t_2-t} \cos \theta_{t_2-t} \\ y = -b\theta_{t_2-t} \sin \theta_{t_2-t} \end{cases} \quad t > \frac{\alpha r_1 + \beta r_2}{v_0} = t_2
 \end{cases} \quad (41)$$

尽管我们已经得出了龙头的具体运动方程，但在新的路径曲线，尤其是 S 弯与螺旋线交界处，精确构建参数化的几何递推关系变得极为复杂。为此，我们采用了数值方法，将问题转化为在已知某节点坐标的情况下，在指定方向上逐步逼近，直到找到满足距离条件的临近节点位置。

5.5.2 数值逼近递推关系

对于在复杂曲线上逼近点的问题，我们可以结合牛顿法与曲线的弧长参数化方法，在保证精度的前提下，通过迭代逐步逼近解。将路径的参数方程(41)记为 $C(t)$ ：

$$C(t) = (x(t), y(t)) \quad (42)$$

已知曲线上一点 $P_i = C(t_i)$ ，我们希望找到 t_{i+1} ，使其满足以下距离条件：

$$\sqrt{(x(t_{i+1}) - x(t_i))^2 + (y(t_{i+1}) - y(t_i))^2} = l_0 - 2l_1 \quad (43)$$

为此，我们定义目标函数 $f(t)$ ，并通过牛顿法迭代逼近 t ，使 $f(t)$ 值为零：

$$f(t) = \sqrt{(x(t) - x(t_i))^2 + (y(t) - y(t_i))^2} - (l_0 - 2l_1) \quad (44)$$

在每次迭代中， t 值的更新基于以下公式：

$$t_{n+1} = t_n - \frac{f(t_n)}{f'(t_n)} \quad (45)$$

其中， $f'(t_n)$ 是目标函数在 t_n 处的导数，其表达式为：

$$f'(t) = \frac{(x(t) - x(t_i))x'(t) + (y(t) - y(t_i))y'(t)}{\sqrt{(x(t) - x(t_i))^2 + (y(t) - y(t_i))^2}} \quad (46)$$

牛顿法在每次迭代中不断更新 t_n ，直至满足**收敛条件**：

$$|f(t_n)| < \varepsilon \quad (47)$$

其中， ε 是精度阈值，取值为 10^{-3}m 。当该条件满足时，我们认为 t_n 已经收敛到 t_{i+1} 。得到各节点位置后，我们可以通过**前向差分法**计算速度各节点速度。

5.5.3 龙头速度规划

根据题目要求，我们需要找到最大龙头速度，使得所有其他节点在带动下速度均不大于 2 m/s 。我们在一系列不同的龙头速度 v_0 下，运行**仿真模型**。通过应用**二分查找**，准确求得龙头在满足速度约束条件的范围内最大化速度。最终得到允许的**最大龙头速度**为 **1.3124 m/s**。此速度下的仿真结果如图所示：

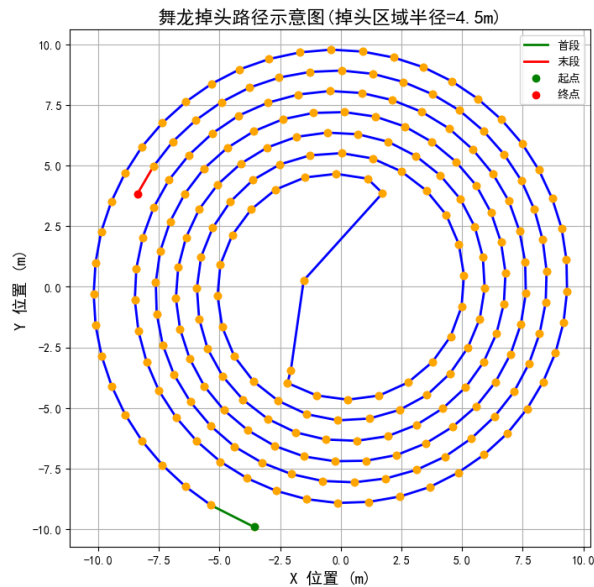


图 10：半径为 4.5m 时舞龙队调头路径示意图

为了更详细地分析节点速度的变化情况，我们在零时刻下绘制了各节点速度的分布，如下图所示：

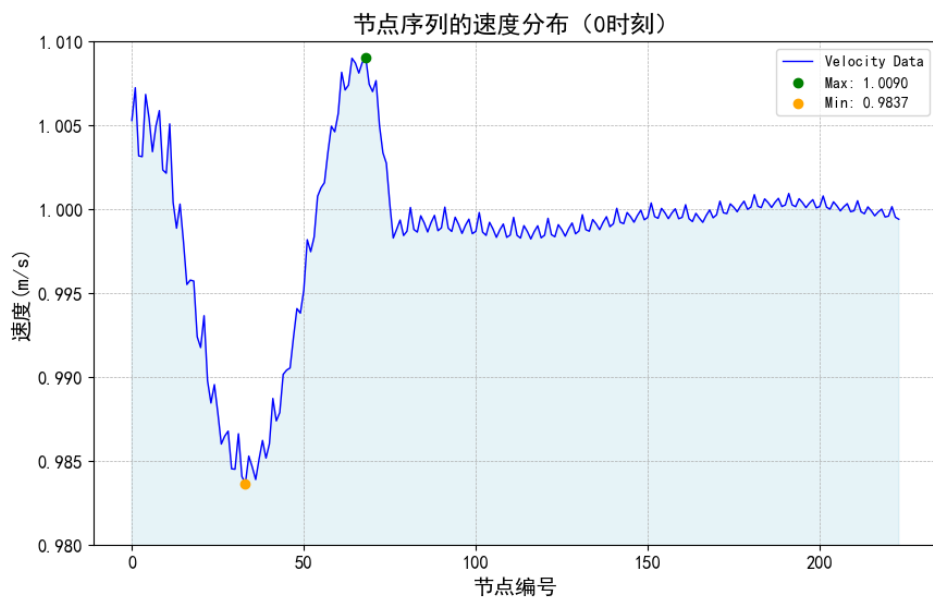


图 11：零时刻舞龙队各点速度分布图

六、 模型分析

6.1 模型检验

在该题中，板凳龙节点的硬性约束是各把手之间的距离保持为定值。在运动过程中，各节点之间的距离应接近板凳的长度：2.85 m 和 1.65 m。

我们对计算结果中各相邻节点间的距离进行了统计分析，并分别执行了 **Shapiro-Wilk 正态性检验**和**单样本 t 检验**，同时计算了方差。统计结果显示，距离数据的方差为 $s = 0.0017 \text{ m}^2$ ，服从正态分布， t 统计量为 1.26， p 值为 0.207（大于 0.05），因此可以认为距离均值与板凳长度之间不存在显著差异，且方差极小。由此可见，本论文建立模型正确。

6.2 结果可靠性检验

考虑到龙头速度测量可能存在误差，故针对问题五中**仿真模型**，每时刻在龙头速度上叠加标准差 **0.3%的高斯噪声**，分别进行仿真，并计算绝对偏差。

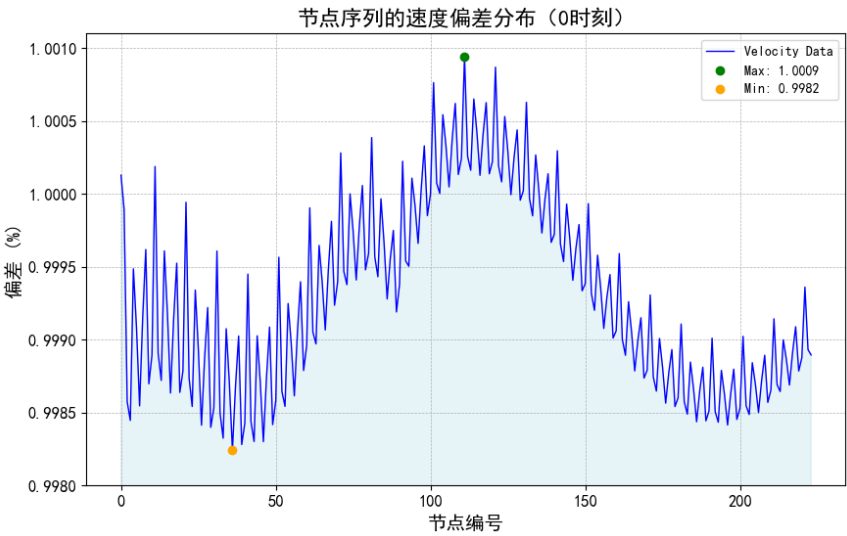


图 12：零时刻舞龙队各点速度偏差分布图

可以看到扰动造成的**绝对偏差**在 **1.001%**以内，故模型求解结果可靠。

七、 模型总结

7.1 模型优点

- 通过微分方程结合**四阶龙格库塔法**求解龙头速度，并基于几何关系递推各节点位置，结果精确度高，效果理想。
- 引入**自适应插值查找算法**、**Armijo 线搜索算法**等手段，提高了模型的计算效率。

7.2 模型缺点

- 动态检测算法复杂度高，查找时运行时间较长，为缩短计算时间导致精度下降。

7.3 模型推广与改进

- 考虑在模型中引入随机性因素或考虑外界环境的变化，例如加入**外部干扰**活运动的不稳定性，使模型在不确定条件下仍能获得**鲁棒性强**的解决方案。

八、 参考文献

- [1] 韦华丽. 北泉板凳龙文化的现状与传承[D]. 西南大学, 2012.
- [2] 冯建强, 孙诗一. 四阶龙格—库塔法的原理及其应用[J]. 数学学习与研究, 2017, (17):3-5.
- [3] 叶三星, 高伟, 古富强, 等. 基于插值预测的快速查找算法[J]. 软件导刊, 2011, 10(11):63-65.

附录

第一部分：支撑材料

支撑材料	
源代码	✓ 代码 1.ipynb 2.ipynb 3.ipynb 45.ipynb
数据	enhanced_velocity_plot.png result1.xlsx result2.xlsx result4.xlsx

第二部分：代码

代码 1: 得到龙头幅角随时间变化及变化率图

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

b = 0.55 / (2 * np.pi)
n = 16

def ditui(a, c):
    left = 4*b*b*c*c*(np.sin((a-c)/2))**2 + b*b*(a-c)**2 + 4*b*b*c*(a-c)*(np.sin((a-c)/2))**2
    return left

# Constants and initial conditions
a = 1 / b # you can modify 'a' as needed
y0 = 2*n*np.pi # initial value of y
x_span = [0, 440] # the range of x values

# Define the differential equation dy/dx = -a / sqrt(y^2 + 1)
def ode_system(x, y):
    return -a / np.sqrt(y**2 + 1)

```

```

# Solve the ODE using solve_ivp
solution = solve_ivp(ode_system, x_span, [y0], t_eval=np.linspace(x_span[0],
x_span[1], 440))
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 创建图像
fig, ax1 = plt.subplots(figsize=(10, 6))

# 绘制主曲线 (y 随 x 变化)
ax1.plot(solution.t, solution.y[0], label='y (蓝色)', color='blue', linewidth=2)
ax1.set_xlabel('x', fontsize=14)
ax1.set_ylabel('y', fontsize=14, color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# 添加填充区域
ax1.fill_between(solution.t, solution.y[0], color='lightblue', alpha=0.3)

# 计算 y 随 x 的变化率 dy/dx
dy_dx = np.gradient(solution.y[0], solution.t)

# 创建第二个 y 轴并绘制变化率曲线 (dy/dx)
ax2 = ax1.twinx() # 共享同一个 x 轴
ax2.plot(solution.t, dy_dx, label='变化率 (dy/dx)', color='orange', linestyle='--',
linewidth=2)
ax2.set_ylabel('变化率 (dy/dx)', fontsize=14, color='orange')
ax2.tick_params(axis='y', labelcolor='orange')

# 标注起点和终点
ax1.scatter(solution.t[0], solution.y[0][0], color='green', label='起点', zorder=5)
ax1.scatter(solution.t[-1], solution.y[0][-1], color='red', label='终点', zorder=5)

# 添加标题
plt.title('龙头辐角随时间变化及变化率图', fontsize=16)

# 添加网格
ax1.grid(True, linestyle='--', alpha=0.7)

# 添加图例
fig.legend(loc='upper right', fontsize=12)
plt.legend()
plt.grid(True)
plt.show()

```


代码 2: 得到 400 秒时舞龙队形状示意图

```

t = solution.t
theta_0 = solution.y[0]
theta = np.zeros((440, 224))
for time_step in range(440):
    theta[time_step][0] = theta_0[time_step]
    up = theta[time_step][0] + np.pi
    down = theta[time_step][0]
    theta[time_step][1] = (up + down)/2
    for iter in range(60):
        left = ditui(theta[time_step][1], theta[time_step][0])
        residual = left - 8.1796
        if residual < 0:
            down = theta[time_step][1]
            theta[time_step][1] = (down + up)/2
        else:
            up = theta[time_step][1]
            theta[time_step][1] = (up + down)/2
    for i in range(1,223):
        iteration = 0
        residual = 100
        # residual_history = np.array([100])
        up = theta[time_step][i] + np.pi
        down = theta[time_step][i]
        theta[time_step][i+1] = (up + down)/2
        for iter in range(60):
            left = ditui(theta[time_step][i+1], theta[time_step][i])
            residual = left - 2.7225
            if residual < 0:
                down = theta[time_step][i+1]
                theta[time_step][i+1] = (down + up)/2
            else:
                up = theta[time_step][i+1]
                theta[time_step][i+1] = (up + down)/2
            # np.append(residual_history, residual)
            # # plt.plot(np.arange(0, iteration), residual_history)
            # # plt.draw()

t_s = 400
x = b * theta[t_s] * np.cos(theta[t_s])
y = b * theta[t_s] * np.sin(theta[t_s])

# 绘制舞龙路径

```

```

plt.figure(figsize=(8, 8))

# 绘制路径线条
plt.plot(x, y, color='blue', linewidth=2, label='舞龙路径')

# 标出路径上的每个点
plt.scatter(x, y, color='red', s=10, zorder=5, label='把手点')

# 添加标题和标签
plt.title('舞龙队形状示意图 (400s)', fontsize=16)
plt.xlabel('X 坐标', fontsize=14)
plt.ylabel('Y 坐标', fontsize=14)

# 设置网格
plt.grid(True, linestyle='--', alpha=0.7)

# 设置等比例轴
plt.axis('equal')

# 显示图例
plt.legend()

# 显示图像
plt.show()

```

代码 3: 得到问题一中要求的位置和速度计算结果, 并将结果存入表格

```

x = b * theta * np.cos(theta)
y = b * theta * np.sin(theta)

with open('result1_1.csv', 'w') as f:
    f.write(' ,0s, 60s, 120s, 180s, 240s, 300s\n')
    f.write('龙头 x (m), '+ str(x[0][0]) +', '+ str(x[60][0]) +', '+ str(x[120][0])
+', '+ str(x[180][0]) +', '+ str(x[240][0]) +', '+ str(x[300][0]) +'\n')
    f.write('龙头 y (m), '+ str(y[0][0]) +', '+ str(y[60][0]) +', '+ str(y[120][0])
+', '+ str(y[180][0]) +', '+ str(y[240][0]) +', '+ str(y[300][0]) +'\n')
    f.write('第 1 节龙身 x (m), '+ str(x[0][1]) +', '+ str(x[60][1]) +', '+
str(x[120][1]) +', '+ str(x[180][1]) +', '+ str(x[240][1]) +', '+ str(x[300][1]) +'\n')
    f.write('第 1 节龙身 y (m), '+ str(y[0][1]) +', '+ str(y[60][1]) +', '+
str(y[120][1]) +', '+ str(y[180][1]) +', '+ str(y[240][1]) +', '+ str(y[300][1]) +'\n')
    f.write('第 51 节龙身 x (m), '+ str(x[0][51]) +', '+ str(x[60][51]) +', '+
str(x[120][51]) +', '+ str(x[180][51]) +', '+ str(x[240][51]) +', '+ str(x[300][51])
+', '+ '\n')

```

```

        f.write('第 51 节龙身 y (m), '+ str(y[0][51]) +', '+ str(y[60][51]) +', '+
str(y[120][51]) +', '+ str(y[180][51]) +', '+ str(y[240][51]) +', '+ str(y[300][51])
+'\n')

        f.write('第 101 节龙身 x (m), '+ str(x[0][101]) +', '+ str(x[60][101]) +',
'+ str(x[120][101]) +', '+ str(x[180][101]) +', '+ str(x[240][101]) +', '+
str(x[300][101]) +'\n')

        f.write('第 101 节龙身 y (m), '+ str(y[0][101]) +', '+ str(y[60][101]) +',
'+ str(y[120][101]) +', '+ str(y[180][101]) +', '+ str(y[240][101]) +', '+
str(y[300][101]) +'\n')

        f.write('第 151 节龙身 x (m), '+ str(x[0][151]) +', '+ str(x[60][151]) +',
'+ str(x[120][151]) +', '+ str(x[180][151]) +', '+ str(x[240][151]) +', '+
str(x[300][151]) +'\n')

        f.write('第 151 节龙身 y (m), '+ str(y[0][151]) +', '+ str(y[60][151]) +',
'+ str(y[120][151]) +', '+ str(y[180][151]) +', '+ str(y[240][151]) +', '+
str(y[300][151]) +'\n')

        f.write('第 201 节龙身 x (m), '+ str(x[0][201]) +', '+ str(x[60][201]) +',
'+ str(x[120][201]) +', '+ str(x[180][201]) +', '+ str(x[240][201]) +', '+
str(x[300][201]) +'\n')

        f.write('第 201 节龙身 y (m), '+ str(y[0][201]) +', '+ str(y[60][201]) +',
'+ str(y[120][201]) +', '+ str(y[180][201]) +', '+ str(y[240][201]) +', '+
str(y[300][201]) +'\n')

        f.write('龙尾（后） x (m), '+ str(x[0][-1]) +', '+ str(x[60][-1]) +', '+
str(x[120][-1]) +', '+ str(x[180][-1]) +', '+ str(x[240][-1]) +', '+ str(x[300][-1]) +'\n')

        f.write('龙尾（后） y (m), '+ str(y[0][-1]) +', '+ str(y[60][-1]) +', '+
str(y[120][-1]) +', '+ str(y[180][-1]) +', '+ str(y[240][-1]) +', '+ str(y[300][-1]) +'\n')

velocity = np.zeros((7, 6))
time_list = np.linspace(0, 300, 6)
num_list = [0, 1, 51, 101, 151, 201, -1]
for i in range(6):
    for j in range(1, 7):
        ts = int(time_list[i])
        ni = int(num_list[j])

        vx = (x[ts+1][ni] - x[ts-1][ni])/2
        vy = (y[ts+1][ni] - y[ts-1][ni])/2
        vel = np.sqrt(vx**2 + vy**2)

        velocity[j][i] = vel

velocity[0] = 1
with open('result1_2.csv', 'w') as f:
    f.write(' ,0s, 60s, 120s, 180s, 240s, 300s\n')

```

```

f.write('龙头 (m/s), '+str(velocity[0][0])+', '+str(velocity[0][1])+',
'+str(velocity[0][2])+', '+str(velocity[0][3])+', '+str(velocity[0][4])+',
'+str(velocity[0][5])+', '\n')
f.write('第 1 节龙身 (m/s), '+str(velocity[1][0])+', '+str(velocity[1][1])+',
'+str(velocity[1][2])+', '+str(velocity[1][3])+', '+str(velocity[1][4])+',
'+str(velocity[1][5])+', '\n')
f.write('第 51 节龙身 (m/s), '+str(velocity[2][0])+', '+str(velocity[2][1])+',
'+str(velocity[2][2])+', '+str(velocity[2][3])+', '+str(velocity[2][4])+',
'+str(velocity[2][5])+', '\n')
f.write('第 101 节龙身 (m/s), '+str(velocity[3][0])+', '+str(velocity[3][1])+',
'+str(velocity[3][2])+', '+str(velocity[3][3])+', '+str(velocity[3][4])+',
'+str(velocity[3][5])+', '\n')
f.write('第 151 节龙身 (m/s), '+str(velocity[4][0])+', '+str(velocity[4][1])+',
'+str(velocity[4][2])+', '+str(velocity[4][3])+', '+str(velocity[4][4])+',
'+str(velocity[4][5])+', '\n')
f.write('第 201 节龙身 (m/s), '+str(velocity[5][0])+', '+str(velocity[5][1])+',
'+str(velocity[5][2])+', '+str(velocity[5][3])+', '+str(velocity[5][4])+',
'+str(velocity[5][5])+', '\n')
f.write('龙尾 (m/s), '+str(velocity[6][0])+', '+str(velocity[6][1])+',
'+str(velocity[6][2])+', '+str(velocity[6][3])+', '+str(velocity[6][4])+',
'+str(velocity[6][5])+', '\n')

```

代码 4: 动态碰撞检测判断

```

import numpy as np
import matplotlib.pyplot as plt

p = 0.45
b = p / (2 * np.pi)
n = 16
numt = 810

def ditui(a, c):
    left = 4*b*b*c*c*(np.sin((a-c)/2))**2 + b*b*(a-c)**2 + 4*b*b*c*(a-
c)*(np.sin((a-c)/2))**2
    return left

from scipy.integrate import solve_ivp

# Constants and initial conditions
a = 1 / b # you can modify 'a' as needed
y0 = 2*n*np.pi # initial value of y
x_span = [0, numt] # the range of x values

```

```

# Define the differential equation  $dy/dx = -a / \sqrt{y^2 + 1}$ 
def ode_system(x, y):
    return -a / np.sqrt(y**2 + 1)

# Solve the ODE using solve_ivp
solution = solve_ivp(ode_system, x_span, [y0], t_eval=np.linspace(x_span[0],
x_span[1], numt))

t = solution.t
theta_0 = solution.y[0]
theta = np.zeros((810, 224))
for time_step in range(810):
    theta[time_step][0] = theta_0[time_step]
    up = theta[time_step][0] + np.pi
    down = theta[time_step][0]
    theta[time_step][1] = (up + down)/2
    for iter in range(60):
        left = ditui(theta[time_step][1], theta[time_step][0])
        residual = left - 8.1796
        if residual < 0:
            down = theta[time_step][1]
            theta[time_step][1] = (down + up)/2
        else:
            up = theta[time_step][1]
            theta[time_step][1] = (up + down)/2
    for i in range(1,223):
        iteration = 0
        residual = 100
        # residual_history = np.array([100])
        up = theta[time_step][i] + np.pi
        down = theta[time_step][i]
        theta[time_step][i+1] = (up + down)/2
        for iter in range(60):
            left = ditui(theta[time_step][i+1], theta[time_step][i])
            residual = left - 2.7225
            if residual < 0:
                down = theta[time_step][i+1]
                theta[time_step][i+1] = (down + up)/2
            else:
                up = theta[time_step][i+1]
                theta[time_step][i+1] = (up + down)/2

# 判断两个矩形是否发生碰撞

```

```

def check_collision(rect1, rect2):
    # 使用分离轴定理（SAT）检测两矩形是否重叠
    def get_axes(rect):
        # 获取矩形的两条边向量（平行于边）
        edges = [
            (rect[1][0] - rect[0][0], rect[1][1] - rect[0][1]),
            (rect[3][0] - rect[0][0], rect[3][1] - rect[0][1])
        ]
        # 返回垂直于边的法向量（即潜在的分离轴）
        axes = [(-edges[0][1], edges[0][0]), (-edges[1][1], edges[1][0])]
        return axes

    def project_polygon(axis, rect):
        # 将矩形投影到指定轴上，返回投影的最小值和最大值
        projections = [(vertex[0] * axis[0] + vertex[1] * axis[1]) for vertex in
rect]

        return min(projections), max(projections)

    def overlap(proj1, proj2):
        # 检查两个投影是否重叠
        return proj1[1] >= proj2[0] and proj2[1] >= proj1[0]

    # 获取两个矩形的法向量（潜在分离轴）
    axes1 = get_axes(rect1)
    axes2 = get_axes(rect2)
    axes = axes1 + axes2

    # 检查所有轴上的投影是否有重叠
    for axis in axes:
        proj1 = project_polygon(axis, rect1)
        proj2 = project_polygon(axis, rect2)
        if not overlap(proj1, proj2):
            return False # 如果没有重叠，说明分离轴存在，矩形不
重叠

    return True # 所有轴上的投影都有重叠，说明矩形重叠

# 构建质点间的碰撞箱（长方形）
def build_collision_box(x, y, i, j):
    # 相邻两个质点的坐标
    p1 = np.array([x[i], y[i]])
    p2 = np.array([x[j], y[j]])

    # 长 220cm，宽 30cm，质点在长轴两端，距离短边边界 27.5cm
    length = 2.20 # cm

```

```

width = 0.3      # cm
offset = 0.275   # cm

# 计算质点连线的单位向量
direction = (p2 - p1) / np.linalg.norm(p2 - p1)

# 计算与质点连线垂直的单位法向量，用于构建短边方向
normal = np.array([-direction[1], direction[0]])

# 构建矩形的四个顶点，先计算从质点偏移到碰撞箱边缘的两个点
p1_left = p1 - direction * offset + normal * (width / 2)
p1_right = p1 - direction * offset - normal * (width / 2)
p2_left = p2 + direction * offset + normal * (width / 2)
p2_right = p2 + direction * offset - normal * (width / 2)

# 返回碰撞箱的四个顶点
return [p1_left, p1_right, p2_right, p2_left]

# 检测碰撞的时间步
def detect_collision_time(x, y):
    num_particles = 223
    num_timesteps = 810

    for t in range(num_timesteps):
        # 构建当前时间步下的所有相邻质点的碰撞箱
        collision_boxes = []
        for i in range(num_particles - 1):
            collision_boxes.append(build_collision_box(x[t], y[t], i, i + 1))

        # 检测非相邻的碰撞箱之间是否有碰撞
        for i in range(len(collision_boxes)):
            for j in range(i + 2, len(collision_boxes)): # 非相邻
                if check_collision(collision_boxes[i], collision_boxes[j]):
                    return t # 返回第一次检测到碰撞的时间步
            for j in range(0, i-1): # 非相邻
                if check_collision(collision_boxes[i], collision_boxes[j]):
                    return t
    return -1 # 如果没有发生碰撞

# 示例质点运动数据
x = b * theta * np.cos(theta)
y = b * theta * np.sin(theta)

# 检测碰撞发生的时间步

```

```
collision_time = detect_collision_time(x, y)
```

代码 5: 计算碰撞发生和到达掉头区域边界的时间

```
def signal(p):
    b = p/(2*np.pi)
    def ode_system(x, y):
        return -b / np.sqrt(y**2 + 1)

    # Solve the ODE using solve_ivp
    solution = solve_ivp(ode_system, x_span, [y0],
t_eval=np.linspace(x_span[0], x_span[1], 810))

    theta_stop = 9 / b
    time_stop = np.argmin(np.abs(solution.y[0] - theta_stop))
    theta_0 = solution.y[0]

    theta = np.zeros((810, 224))
    for time_step in range(810):
        theta[time_step][0] = theta_0[time_step]
        up = theta[time_step][0] + np.pi
        down = theta[time_step][0]
        theta[time_step][1] = (up + down)/2
        for iter in range(60):
            left = ditui(theta[time_step][1], theta[time_step][0])
            residual = left - 8.1796
            if residual < 0:
                down = theta[time_step][1]
                theta[time_step][1] = (down + up)/2
            else:
                up = theta[time_step][1]
                theta[time_step][1] = (up + down)/2
        for i in range(1,223):
            residual = 100
            # residual_history = np.array([100])
            up = theta[time_step][i] + np.pi
            down = theta[time_step][i]
            theta[time_step][i+1] = (up + down)/2
            while np.abs(residual) > 1e-7:
                left = ditui(theta[time_step][i+1], theta[time_step][i])
                residual = left - 2.7225
                if residual < 0:
```



```

        down = theta[time_step][i+1]
        theta[time_step][i+1] = (down + up)/2
    else:
        up = theta[time_step][i+1]
        theta[time_step][i+1] = (up + down)/2

    x = b * theta * np.cos(theta)
    y = b * theta * np.sin(theta)

    collision_time = detect_collision_time(x, y)
    return collision_time, time_stop

```

代码 6: 求解问题三中最小螺距

```

import numpy as np
import matplotlib.pyplot as plt

def objective(x1, x2, r):
    k = 2/np.pi
    y1 = k*x1
    y2 = k*x2
    vec_1 = np.array([k, 1])
    vec_2 = np.array([x2-x1, y2-y1])
    cosine = np.dot(vec_1,
vec_2)/(np.linalg.norm(vec_1)*np.linalg.norm(vec_2))
    angle = np.arccos(cosine)
    radius = np.linalg.norm(vec_2)/3

    d = 1.7
    cta_0 = 9*np.pi/d
    cta = 2*np.pi*r/d
    log_numerator = np.sqrt(cta_0**2+1) + cta_0
    log_denominator = np.sqrt(cta**2+1) + cta
    length = angle*4.5 + (d/np.pi) * (np.log(log_numerator/log_denominator)
+ cta_0*np.sqrt(cta_0**2+1) - cta*np.sqrt(cta**2+1)) / 2
    return length

# 使用数值方法进行求解
import numpy as np
from scipy.optimize import fsolve

# 定义方程组
def equations(vars, r):

```

```

x, y = vars
a = (2/np.pi)*x + r
b = (2/np.pi)*y - r
eq1 = x**2 + (r - a)**2 - ((y - x)**2 + (a - b)**2)/9
eq2 = y**2 + (r + b)**2 - ((y - x)**2 + (a - b)**2)*4/9
return [eq1, eq2]

# 生成 S 弯圆半径 的取值范围
r_values = np.linspace(0, 4.5, 100)

# 存储 x, y 的解
solutions = np.zeros((3, 100))

# 对每个 a 进行求解
for i in range(len(r_values)):
    r_val = r_values[i]
    # 初始猜测值
    initial_guess = [0, 0]
    # 使用 fsolve 进行数值求解
    sol = fsolve(equations, initial_guess, args=(r_val))
    solutions[2][i] = r_val
    solutions[0][i] = sol[0]
    solutions[1][i] = sol[1]

x_center_1 = solutions[0]
x_center_2 = solutions[1]
r_list = solutions[2]
# 找到掉头路径长度的最小值及其对应的半径
min_index = np.argmin(l)
min_r = r_list[min_index]
min_length = l[min_index]

# 画出优化后的图像
plt.plot(r_list, l, color='blue', linewidth=2, label='掉头路径的长度') # 添加
图例
plt.fill_between(r_list, l, color='lightblue', alpha=0.5) # 添加曲线下方的填
充颜色
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False
# 标记掉头路径长度最小值并画垂直线
plt.scatter(min_r, min_length, color='red', zorder=5) # 用红点标记最小值
plt.axvline(x=min_r, color='red', linestyle=':', linewidth=2) # 画垂直线
plt.text(min_r, min_length, f'最小值\n半径: {min_r:.2f}', fontsize=10,
verticalalignment='bottom', horizontalalignment='right') # 标注

```

```

# 设置轴标签和标题
plt.xlabel('半径', fontsize=12) # 设置横轴标签
plt.ylabel('掉头路径的长度', fontsize=12) # 设置纵轴标签
plt.title('半径与掉头路径长度的关系图', fontsize=14) # 设置图表标题

# 添加网格线和图例
plt.grid(True, linestyle=':', color='gray', alpha=0.7) # 添加网格线
plt.legend(fontsize=10) # 显示图例
plt.tight_layout() # 优化图像布局
plt.show()

```

代码 7: 得到问题四中要求的位置和速度计算结果, 并将结果存入表格

```

v = 1
radius_direction = np.array([2/np.pi, 1])
center_direction = np.array([])
x1 = -1.5*radius_direction[0]
y1 = 4.5-1.5*radius_direction[1]
x2 = 3*radius_direction[0]
y2 = -4.5+3*radius_direction[1]
x1 = -0.7456*x1-0.6664*y1
y1 = 0.6664*x1-0.7456*y1
x2 = -0.7456*x2-0.6664*y2
y2 = 0.6664*x2-0.7456*y2
center_direction = np.array([x2-x1, y2-y1])

b = 1.7/(2*np.pi)
a = 1 / b # you can modify 'a' as needed
y0 = 2*16*np.pi # initial value of y
x_span = [0, 3000] # the range of x values

# Define the differential equation dy/dx = -a / sqrt(y^2 + 1)
def ode_system(x, y):
    return -a / np.sqrt(y**2 + 1)

# Solve the ODE using solve_ivp
solution = solve_ivp(ode_system, x_span, [y0], t_eval=np.linspace(x_span[0],
x_span[1], 3000))
theta_0 = solution.y[0]
t_in = np.argmin(np.abs(theta_0 - 4.5/b))

```

```

gamma = np.arccos(np.dot(radius_direction,
center_direction)/(np.linalg.norm(center_direction)*np.linalg.norm(radius_directio
n)))
t_shift = t_in + int(np.round(1.5*gamma))
x_in = b*theta_0[t_in]*np.cos(theta_0[t_in])
y_in = b*theta_0[t_in]*np.sin(theta_0[t_in])
t_out = t_shift + int(np.round(3*gamma))
x_out = -x_in
y_out = -y_in
x_shift = 2*x1/3 + x2/3
y_shift = 2*y1/3 + y2/3

def curve_function(t):
    if t <= t_in:
        x = b*theta_0[t]*np.cos(theta_0[t])
        y = b*theta_0[t]*np.sin(theta_0[t])
        return [x, y]
    elif t <= t_shift:
        alpha = np.arctan(np.pi/2) - v*t/1.5
        x = x1 + 1.5*np.cos(alpha)
        y = y1 + 1.5*np.sin(alpha)
        return [x, y]
    elif t <= t_out:
        beta = np.arctan((y1 - y2)/(x2 - x1)) - v*t/3
        x = x2 - 3*np.cos(beta)
        y = y2 + 3*np.sin(beta)
        return [x, y]
    else:
        t_index = t_in + t_out - t
        x = -b*theta_0[t_index]*np.cos(theta_0[t_index])
        y = -b*theta_0[t_index]*np.sin(theta_0[t_index])
        return [x, y]

def curve_location(ti):
    down = int(np.floor(ti))
    up = down + 1
    weight = ti-down
    [loc_1x, loc_1y] = curve_function(down)
    [loc_2x, loc_2y] = curve_function(up)
    loc_1 = np.array([loc_1x, loc_1y])
    loc_2 = np.array([loc_2x, loc_2y])
    loc = (1-weight)*loc_1 + weight*loc_2
    return loc

t_diff_list = -np.linspace(0, 222*1.3, 223) - 2

```

```

t_diff_list = np.insert(t_diff_list, 0, 0)/v
x_cor = np.zeros((t_in+t_out, 224))
y_cor = np.zeros((t_in+t_out, 224))

for t in range(t_in+t_out):
    time_list = t + t_diff_list
    for i in range(len(time_list)):
        [x_loc, y_loc] = curve_location(time_list[i])
        x_cor[t][i] = x_loc
        y_cor[t][i] = y_loc
velocity = np.zeros((7, 5))
xx = np.zeros((7, 5))
yy = np.zeros((7, 5))
time_list = [t_in-100, t_in-50, t_in, t_in+50, t_in+100]
num_list = [0, 1, 51, 101, 151, 201, -2]
for i in range(len(time_list)):
    for j in range(len(num_list)):
        ts = time_list[i]
        ni = num_list[j]

        vx = x_cor[ts+1][ni] - x_cor[ts][ni]
        vy = y_cor[ts+1][ni] - y_cor[ts][ni]
        vel = np.sqrt(vx**2 + vy**2)

        velocity[j][i] = vel
        xx[j][i] = x_cor[ts][ni]
        yy[j][i] = y_cor[ts][ni]

with open('result4_0.csv', 'w') as f:
    f.write(' ,-100s, -50s, 0s, 50s, 100s\n')
    f.write('龙头 (m/s), '+str(velocity[0][0])+', '+str(velocity[0][1])+',
'+str(velocity[0][2])+', '+str(velocity[0][3])+', '+str(velocity[0][4])+', \n')
    f.write('第 1 节龙身 (m/s), '+str(velocity[1][0])+',
'+str(velocity[1][1])+', '+str(velocity[1][2])+', '+str(velocity[1][3])+',
'+str(velocity[1][4])+', \n')
    f.write('第 51 节龙身 (m/s), '+str(velocity[2][0])+',
'+str(velocity[2][1])+', '+str(velocity[2][2])+', '+str(velocity[2][3])+',
'+str(velocity[2][4])+', \n')
    f.write('第 101 节龙身 (m/s), '+str(velocity[3][0])+',
'+str(velocity[3][1])+', '+str(velocity[3][2])+', '+str(velocity[3][3])+',
'+str(velocity[3][4])+', \n')
    f.write('第 151 节龙身 (m/s), '+str(velocity[4][0])+',
'+str(velocity[4][1])+', '+str(velocity[4][2])+', '+str(velocity[4][3])+',
'+str(velocity[4][4])+', \n')

```

```

        f.write('第 201 节龙身 (m/s), '+str(velocity[5][0])+',
'+str(velocity[5][1])+', '+str(velocity[5][2])+', '+str(velocity[5][3])+',
'+str(velocity[5][4])+', \n')
        f.write('龙尾 (m/s), '+str(velocity[6][0])+', '+str(velocity[6][1])+',
'+str(velocity[6][2])+', '+str(velocity[6][3])+', '+str(velocity[6][4])+', \n')
        f.write('龙头 x (m), '+str(xx[0][0])+', '+str(xx[0][1])+', '+str(xx[0][2])+',
'+str(xx[0][3])+', '+str(xx[0][4])+', \n')
        f.write('第 1 节龙身 x (m), '+str(xx[1][0])+', '+str(xx[1][1])+',
'+str(xx[1][2])+', '+str(xx[1][3])+', '+str(xx[1][4])+', \n')
        f.write('第 51 节龙身 x (m), '+str(xx[2][0])+', '+str(xx[2][1])+',
'+str(xx[2][2])+', '+str(xx[2][3])+', '+str(xx[2][4])+', \n')
        f.write('第 101 节龙身 x (m), '+str(xx[3][0])+', '+str(xx[3][1])+',
'+str(xx[3][2])+', '+str(xx[3][3])+', '+str(xx[3][4])+', \n')
        f.write('第 151 节龙身 x (m), '+str(xx[4][0])+', '+str(xx[4][1])+',
'+str(xx[4][2])+', '+str(xx[4][3])+', '+str(xx[4][4])+', \n')
        f.write('第 201 节龙身 x (m), '+str(xx[5][0])+', '+str(xx[5][1])+',
'+str(xx[5][2])+', '+str(xx[5][3])+', '+str(xx[5][4])+', \n')
        f.write('龙尾 x (m), '+str(xx[6][0])+', '+str(xx[6][1])+', '+str(xx[6][2])+',
'+str(xx[6][3])+', '+str(xx[6][4])+', \n')
        f.write('龙头 y (m), '+str(yy[0][0])+', '+str(yy[0][1])+', '+str(yy[0][2])+',
'+str(yy[0][3])+', '+str(yy[0][4])+', \n')
        f.write('第 1 节龙身 y (m), '+str(yy[1][0])+', '+str(yy[1][1])+',
'+str(yy[1][2])+', '+str(yy[1][3])+', '+str(yy[1][4])+', \n')
        f.write('第 51 节龙身 y (m), '+str(yy[2][0])+', '+str(yy[2][1])+',
'+str(yy[2][2])+', '+str(yy[2][3])+', '+str(yy[2][4])+', \n')
        f.write('第 101 节龙身 y (m), '+str(yy[3][0])+', '+str(yy[3][1])+',
'+str(yy[3][2])+', '+str(yy[3][3])+', '+str(yy[3][4])+', \n')
        f.write('第 151 节龙身 y (m), '+str(yy[4][0])+', '+str(yy[4][1])+',
'+str(yy[4][2])+', '+str(yy[4][3])+', '+str(yy[4][4])+', \n')
        f.write('第 201 节龙身 y (m), '+str(yy[5][0])+', '+str(yy[5][1])+',
'+str(yy[5][2])+', '+str(yy[5][3])+', '+str(yy[5][4])+', \n')
        f.write('龙尾 y (m), '+str(yy[6][0])+', '+str(yy[6][1])+', '+str(yy[6][2])+',
'+str(yy[6][3])+', '+str(yy[6][4])+', \n')
        velocity = np.zeros((224, 201))
        for i in range(224):
            for j in range(201):
                tlist = np.arange(-100, 101)+t_in
                t = tlist[j]
                vx = x_cor[t+1][i] - x_cor[t][i]
                vy = y_cor[t+1][i] - y_cor[t][i]
                vel = np.sqrt(vx**2 + vy**2)

                velocity[i][j] = vel

```

```

with open('result4_2.csv', 'w') as f:
    for i in range(224):
        for j in range(201):
            f.write(str(velocity[i][j])+',')
        f.write('\n')

with open('result4_1.csv', 'w') as f:
    for i in range(224):
        for j in range(201):
            f.write(str(x_cor[i][j])+',')
        f.write('\n')
        for j in range(201):
            f.write(str(y_cor[i][j])+',')
        f.write('\n')

```

代码 8: 得到 0 时刻各节点序列速度分布

```

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 解决坐标轴负号显示问题
# 假设 velocity 数据已经生成, 直接使用你提供的数据进行绘制
velocity_data = velocity[:, 99]
time = np.arange(0, len(velocity_data))

# 查找并标注最大值和最小值
max_index = np.argmax(velocity_data)
min_index = np.argmin(velocity_data)
max_velocity = velocity_data[max_index]
min_velocity = velocity_data[min_index]

# 开始绘制图像
plt.figure(figsize=(10, 6))

# 绘制原始数据曲线
plt.plot(time, velocity_data, color='blue', linewidth=1, label='速度数据')

# 标注最大值和最小值
plt.scatter(max_index, max_velocity, color='green', label=f'Max:
{max_velocity:.4f}', zorder=5)
plt.scatter(min_index, min_velocity, color='orange', label=f'Min:
{min_velocity:.4f}', zorder=5)

# 填充曲线下方的区域

```

```

plt.fill_between(time, velocity_data, color='lightblue', alpha=0.3)

# 添加标题、标签和图例
plt.title('节点序列的速度分布（0时刻）', fontsize=16)
plt.xlabel('节点编号', fontsize=14)
plt.ylabel('速度(m/s)', fontsize=14)

# 设置竖直坐标轴范围
plt.ylim([0.980, 1.010])

# 自定义坐标轴刻度
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.grid(True, which='both', linestyle='--', linewidth=0.5)

# 显示图例
plt.legend()

# 显示图像
plt.show()
f.write('第 151 节龙身 x (m), '+str(xx[4][0])+',
'+str(xx[4][1])+', '+str(xx[4][2])+', '+str(xx[4][3])+', '+str(xx[4][4])+', \n')
f.write('第 201 节龙身 x (m), '+str(xx[5][0])+', '+str(xx[5][1])+',
'+str(xx[5][2])+', '+str(xx[5][3])+', '+str(xx[5][4])+', \n')
f.write('龙尾 x (m), '+str(xx[6][0])+', '+str(xx[6][1])+', '+str(xx[6][2])+',
'+str(xx[6][3])+', '+str(xx[6][4])+', \n')
f.write('龙头 y (m), '+str(yy[0][0])+', '+str(yy[0][1])+', '+str(yy[0][2])+',
'+str(yy[0][3])+', '+str(yy[0][4])+', \n')
f.write('第 1 节龙身 y (m), '+str(yy[1][0])+', '+str(yy[1][1])+',
'+str(yy[1][2])+', '+str(yy[1][3])+', '+str(yy[1][4])+', \n')
f.write('第 51 节龙身 y (m), '+str(yy[2][0])+', '+str(yy[2][1])+',
'+str(yy[2][2])+', '+str(yy[2][3])+', '+str(yy[2][4])+', \n')
f.write('第 101 节龙身 y (m), '+str(yy[3][0])+', '+str(yy[3][1])+',
'+str(yy[3][2])+', '+str(yy[3][3])+', '+str(yy[3][4])+', \n')
f.write('第 151 节龙身 y (m), '+str(yy[4][0])+', '+str(yy[4][1])+',
'+str(yy[4][2])+', '+str(yy[4][3])+', '+str(yy[4][4])+', \n')
f.write('第 201 节龙身 y (m), '+str(yy[5][0])+', '+str(yy[5][1])+',
'+str(yy[5][2])+', '+str(yy[5][3])+', '+str(yy[5][4])+', \n')
f.write('龙尾 y (m), '+str(yy[6][0])+', '+str(yy[6][1])+', '+str(yy[6][2])+',
'+str(yy[6][3])+', '+str(yy[6][4])+', \n')

```